

Actividad en clases: Tensorflow Playground

El propósito de esta actividad es desarrollar intuición y visualizar aspectos básicos de redes neuronales

Parte A: XOR

Navegue a TensorFlow Playground (<https://playground.tensorflow.org/#networkShape=&dataset=xor&discretize=true>), seleccione el dataset XOR. Este dataset se conoce como el problema XOR: las regiones donde $x_1, x_2 > 0$ y $x_1, x_2 < 0$ tienen valor $y = +1$ (puntos azules), y las regiones $x_1 > 0, x_2 > 0, x_1 < 0, x_2 < 0$ tienen valor $y = -1$ (puntos naranjos)

- **Seleccione un modelo lineal, sin capas ocultas, con las features x_1 y x_2 . ¿Se puede ajustar la data con este modelo? ¿Por qué sí o por qué no?**

No se puede ajustar la data con este modelo. Esto ocurre porque el problema XOR no es linealmente separable, ya que los datos de una misma clase se encuentran en esquinas opuestas y no existe una única recta capaz de separar ambas clases.

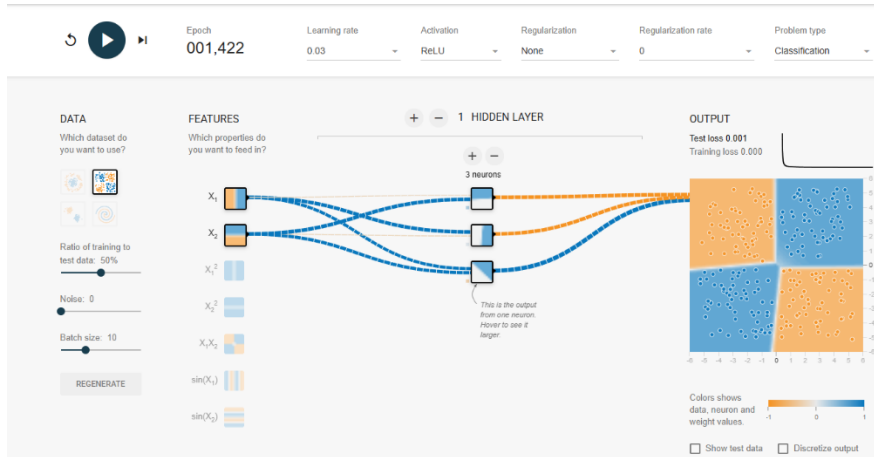
- **Agregue la feature x_1x_2 ¿Qué pasa ahora?**

Al agregar esta nueva variable, el modelo sí puede separar correctamente las clases. El ajuste es exacto y se logra de forma casi inmediata, obteniendo valores de pérdida muy cercanos a cero.

- **Vuelva a dejar solo (x_1, x_2) como features y empiece a agregar capas ocultas. ¿Cuál es la red más chica (menos capas, menos neuronas por capa) que logra ajustar los datos de entrenamiento "perfectamente" (training loss < 0.001)? ¿Cuál es el test loss correspondiente?**

Tras realizar las pruebas, se determinó que la red neuronal más pequeña capaz de resolver el problema con un training loss de 0.000 es de 1 sola capa oculta conformada por 3 neuronas, utilizando la función de activación ReLU. El test loss correspondiente obtenido fue de 0.001.

- **Guarde screenshot + URL de su solución.**



<https://playground.tensorflow.org/#activation=relu&batchSize=10&dataset=xor®Dataset=reg-plane&learningRate=0.03®ularizationRate=0&noise=0&networkShape=3&seed=0.67143&showTestData=false&discretize=false&percTrainData=50&x=true&y=true&xTimesY=false&xSquared=false&ySquared=false&cosX=false&sinX=false&cosY=false&sinY=false&collectStats=false&problem=classification&initZero=false&hideText=false>

Parte B: espiral

Vuelva nuevamente al link, y seleccione el dataset espiral

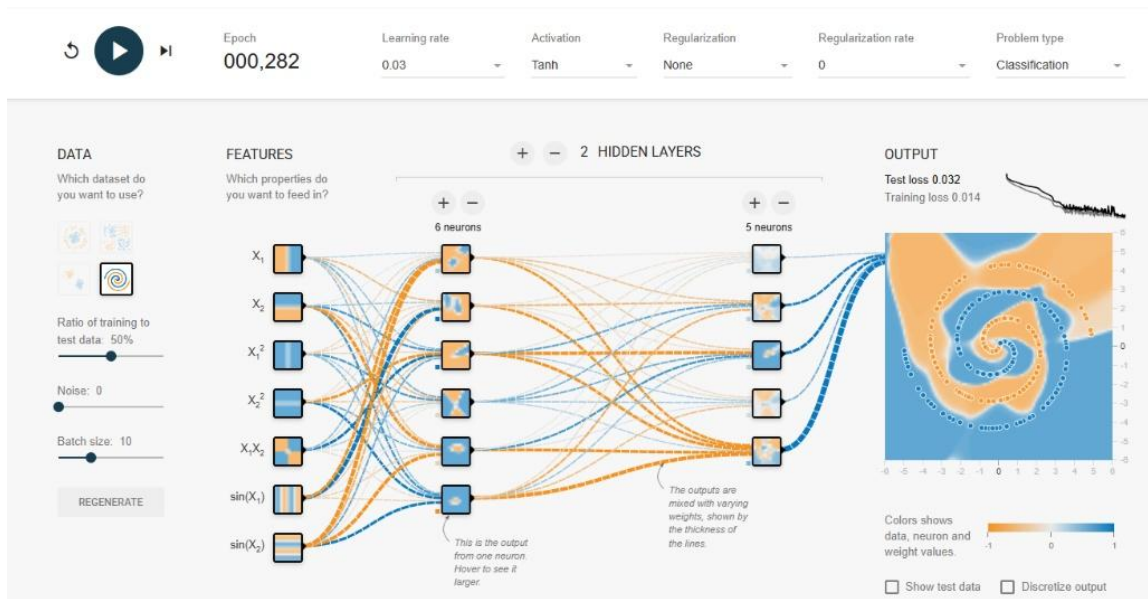
- **Usando todas las features disponibles, encuentre una solución que ajuste los datos de entrenamiento. ¿Qué error de entrenamiento y de test logra? ¿Es un modelo de bias bajo/varianza alta, o bias alto/varianza baja? ¿Cómo lo sabe?**

Al utilizar todas las features disponibles y una arquitectura de 2 capas ocultas, se logró un buen ajuste visual de los datos. El modelo obtuvo un training loss de 0.014 y un test loss de 0.032. Estos resultados indican que se trata de un modelo con bias bajo y varianza alta. Lo sabemos porque el error de entrenamiento es bastante pequeño, demostrando que el modelo aprende bien los datos, pero el error de prueba es más del doble, lo que sugiere que la red se está sobreajustando y pierde capacidad para generalizar.

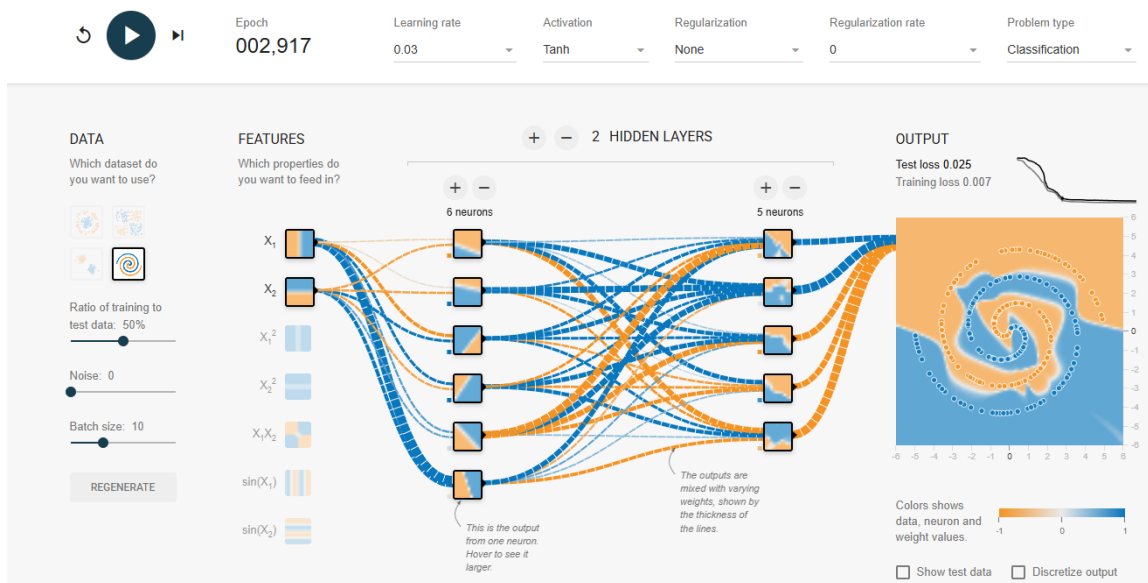
- **Ahora usando solo (x_1, x_2) , encuentre una solución que ajuste los datos de entrenamiento. ¿Qué error de entrenamiento y de test logra? ¿Es bias bajo/varianza alta o bias alto/varianza baja? ¿Cómo lo sabe?**

Utilizando exclusivamente las features x_1 y x_2 , se logró ajustar el modelo mediante una arquitectura de dos capas ocultas. Los errores obtenidos fueron un training loss de 0.007 y un test loss de 0.025. Al evaluar estas métricas, se concluye nuevamente que el modelo presenta un bias bajo y varianza alta. El pequeño error de entrenamiento indica que la red logró capturar la complejidad espacial, pero el error de prueba mayor evidencia sobreajuste al conjunto de entrenamiento.

- Para ambas soluciones, guarden screenshot + URL.



<https://playground.tensorflow.org/#activation=tanh&batchSize=10&dataset=spiral®Dataset=reg-plane&learningRate=0.03®ularizationRate=0&noise=0&networkShape=6,5&seed=0.42049&showTestData=false&discretize=false&percTrainData=50&x=true&y=true&xTimesY=true&xSquared=true&ySquared=true&cosX=false&sinX=true&cosY=false&sinY=true&collectStats=false&problem=classification&initZero=false&hideText=false>



<https://playground.tensorflow.org/#activation=tanh&batchSize=10&dataset=spiral®Dataset=reg-plane&learningRate=0.03®ularizationRate=0&noise=0&networkShape=6,5&seed=0.42049&showTestData=false&discretize=false&percTrainData=50&x=true&y=true&xTimesY=false&xSquared=false&ySquared=false&cosX=false&sinX=false&cosY=false&sinY=false&collectStats=false&problem=classification&initZero=false&hideText=false>

- **Bonus:** Con un modelo lineal (0 capas ocultas), active únicamente 2 de las "engineered" que ofrece Playground (x^2 , x_1^2 , x_1x_2 , $\sin(x_1)$, $\sin(x_2)$) y busque una

combinación que resuelva la espiral con ese modelo lineal. Si la encuentra, guarde screenshot + URL.

Tras probar múltiples combinaciones, se concluye que no existen combinaciones que resuelvan la espiral con ese modelo puramente lineal. Por ejemplo, al activar únicamente $\sin(x_1)$ y $\sin(x_2)$, el modelo no logra converger, obteniendo un test loss de 0.504 y un training loss de 0.499. La espiral tiene una topología muy compleja y combinar de forma lineal solo dos transformaciones simples es matemáticamente insuficiente.

